
Part I: GRPO Finetuning of gemma-3-1b-it on GSM8K

Zixuan Wang (zw499)

Team **chmawa**: sm3035 · bc654 · zw499

I.1 Reproducing the baseline — the run & patches

(i) **The run.** Baseline reproduced as run **R0** ([8c2785ut](#)): the upstream **Tunix / JAX** GRPO pipeline (LoRA on frozen gemma-3-1b-it), commit borisbolliet/tpu-2026@77c5a67, run with the `config.py` defaults — KL-penalty weight $\beta=0.08$, group size $K=2$ (responses sampled per prompt), the full **3364** GRPO steps (one epoch), **~5 h 05 m** wall-clock on v6e-1.

Baseline patches (fixes for what did not run as-shipped; full diff in `tpu-2026_our_changes.patch`):

- TFDS's gsm8k builder crashes under protobuf 7.x once JAX imports — we evaluate the *identical* GSM8K test via an HF datasets fallback (seed 42);
- raised checkpoint retention (`SAVE_INTERVAL_STEPS` 500→100, `MAX_TO_KEEP` 4→40) so the best-val checkpoint survives selection (persistence, not training).

I.1 Reproducing the baseline — accuracy & training curves

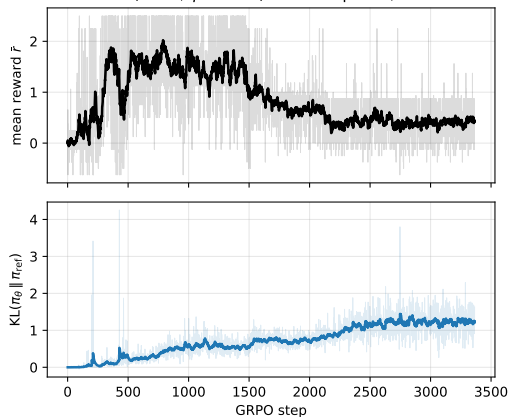
(ii) **Held-out accuracy.** GSM8K *test* (from HF), seed 42, greedy / exact-match, reported as ($n=64$ / $n=1319$) — $n=64$ is the `config.py` default eval, $n=1319$ the full test:

Model / checkpoint	$n=64$	$n=1319$
Base gemma-3-1b-it	45.3	47.4
R0 best-val (step 700)	42.2	46.7
R0 final (step 3364)	0.0	6.2

best-val = the saved checkpoint *nearest the peak held-out validation reward* (not accuracy).

(iii) Training curves (R0)

R0 baseline ($K = 2$, $\beta = 0.08$): reward peaks, then declines



I.2 Understanding the baseline

(a) The three policies (all share the frozen W_0):

- π_θ — LoRA-adapted policy; the **only** trainable policy ($\Delta W = \text{BA}$)
- π_{old} — the rollout policy; with $\mu = \text{NUM_ITERATIONS} = 1$ (one gradient step per rollout batch), $\pi_{\text{old}} = \pi_\theta$ at the update
- π_{ref} — frozen base Gemma

(b) Group size & advantage. $K = \text{NUM_GENERATIONS} = 2$; Tunix 'grp0' is the **standard** group-mean, std-normalised advantage over all K (incl. i) — *not* leave-one-out:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}, \quad \hat{g} = \frac{1}{K} \sum_{i=1}^K h_i \hat{A}_i, \quad h_i = \nabla_\theta \log \pi_\theta(a_i | q).$$

I.2 Understanding the baseline

(c) **Reward = sum of four:**

- **Shaping:** `match_format_exactly`, `match_format_approximately` (the `<reasoning>/<answer>` template)
- **Correctness:** `check_answer` (gold match, only if the template parses) + `check_numbers` (any number after `<answer>`)

With the correctness reward *alone*, σ_r vanishes on most groups — not because the model is rarely right (base = 47%) but because the reward is **coarse**: at $K=2$ the two samples usually *tie*, so

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r} = \frac{0}{\sigma_r + 10^{-6}} = 0.$$

The dense **shaping** terms keep $\sigma_r > 0$ and push toward parseable outputs (the precondition for `check_answer` to vary), although they also invite reward-hacking.

I.2 Understanding the baseline

(d) The clipped surrogate. The PPO-style clip lives in Tunix's GRPOLearner (`tunix.rl`); the clip range is $\epsilon=\text{EPSILON}=0.2$ in `config.py`. **Two deviations** from the lecture form:

(i) Token- vs. sequence-level. The lectures clip a **per-response** ratio (one clip per whole sequence); Tunix clips a **per-token** ratio, token-by-token (a token-level objective):

$$\rho_i = \frac{\pi_\theta(a_i \mid q)}{\pi_{\text{old}}(a_i \mid q)} \quad \text{vs.} \quad \rho_{i,t} = \frac{\pi_\theta(a_{i,t} \mid q, a_{i,<t})}{\pi_{\text{old}}(a_{i,t} \mid q, a_{i,<t})}.$$

(ii) The clip is **inert in this baseline.** With $\mu=\text{NUM_ITERATIONS}=1$ (one gradient step per rollout batch) $\pi_{\text{old}}=\pi_\theta$, so every $\rho \equiv 1$ and **nothing is ever clipped** — the surrogate collapses to the plain policy gradient and ϵ is irrelevant. Not a *contradiction* (the lecture form also reduces to this at $\mu=1$), but the baseline's “clipped” objective does no clipping until $\mu > 1$.

I.3 Why the baseline collapses: gradient variance

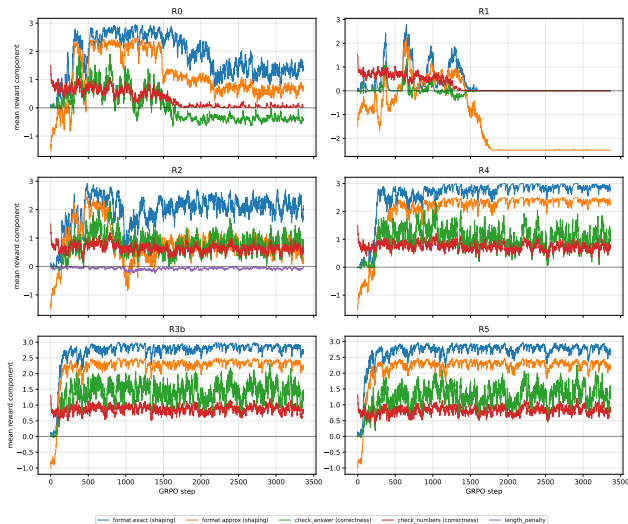
Motivation (grounded in the lecture estimator). The group-mean gradient has variance

$$\text{Var}(\hat{g}) = \frac{1}{K^2} \sum_i (h_i - \bar{h})(h_i - \bar{h})^\top = O\left(\frac{1}{K}\right),$$

- Maximal at the baseline's $K=2 \Rightarrow$ noisiest updates
- Noisy updates let the policy chase the **dense shaping** reward (hack the format) while **true correctness decays**

Prediction to test: raising the group size K shrinks $\text{Var}(\hat{g}) \propto 1/K \Rightarrow$ smoother updates and **milder over-optimisation**.

I.3 The smoking gun: reward decomposition



In **R0**, the **correctness** reward (`check_answer`) **decays** after $\approx$ step 1000 while **format-shaping** **stays pinned high** — textbook reward-hacking.

R1 ($\beta=0.3$) collapses entirely $\approx$ step 1700.

The stable $K=8$ runs (**R3b**, **R5**) keep correctness *high throughout*.

I.3 Two hypotheses, one controlled 2×2

(1) **Variance.** Raise $K \Rightarrow \text{Var}(\hat{g}) \propto 1/K \Rightarrow$ less over-optimisation.

(2) **Regularisation.** The KL penalty pulls toward π_{ref} ; its penalised optimum is

$$\pi^*(a) \propto \pi_{\text{ref}}(a) e^{\hat{A}(a)/\beta} \xrightarrow{\beta \rightarrow \infty} \pi_{\text{ref}},$$

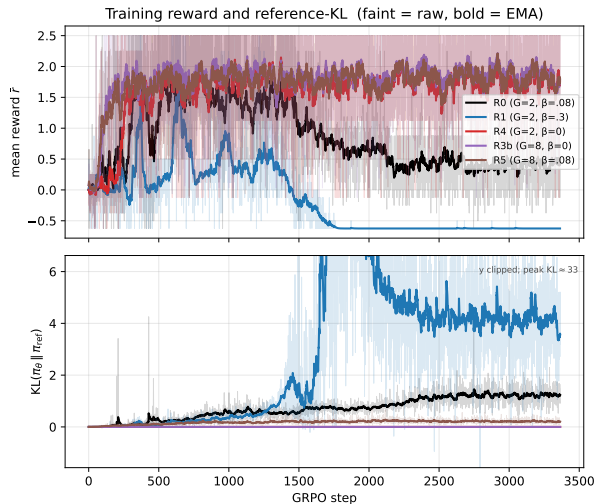
so a *tighter* leash (larger β) should curb the drift.

Design: $\beta \times K$ grid + a β -axis extension

	$\beta=0$	$\beta=0.08$	$\beta=0.3$
$K=2$	R4	R0 (base)	R1
$K=8$	R3b	R5	—

5 full runs, all controlled: same data (HF GSM8K, seed 42), eval, horizon (3364 steps) and rollout RNG (PRNGKey(0)). *Only the named knob changes.*

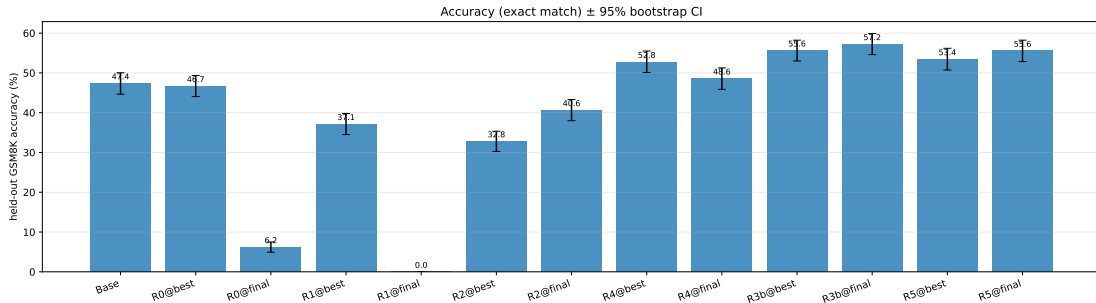
I.3 Training curves: reward and KL (all 5 runs)



- **R0** over-optimises: reward peaks, then declines
- **R1** ($\beta=0.3$) **collapses** — KL blow-up $\approx$ step 1700
- **K=8** (R3b, R5) and R4 ($\beta=0$) stay **stable**

Note: a *higher* β did *not* buy a lower KL or more stability — the opposite of prediction (2).

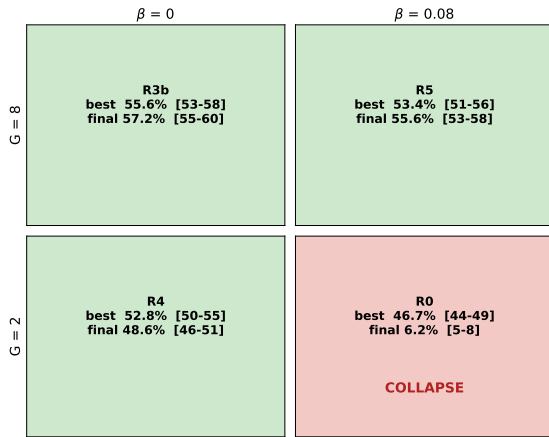
I.3 Held-out accuracy (GSM8K test, $n=1319$)



- **R3b** ($K=8, \beta=0$): **57.2%** final, +9.9 pp over base — best model; paired 95% CI excludes 0
- Both $K=8$ runs beat base; each beats its matched $K=2$ run
- R0 final $\rightarrow$ 6.2%; R1 final $\rightarrow$ 0.0% (below base — a clean negative result)

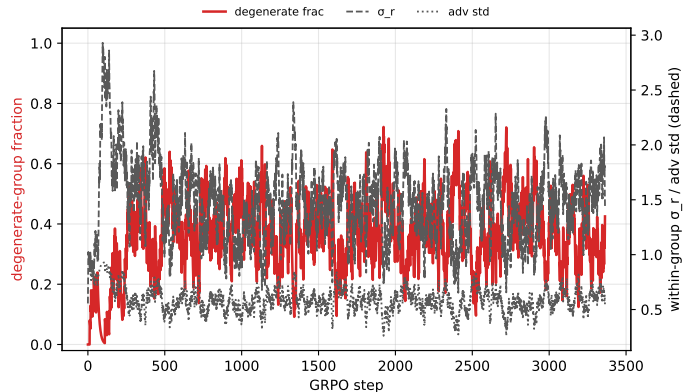
I.3 Collapse lives in one corner: $K=2$, $\beta > 0$

Held-out GSM8K accuracy: group size $G \times$ KL weight β ($n=1319$)
collapse only at $G=2$, $\beta>0$



- Collapse occurs **only** at $K=2$, $\beta > 0$ (R0)
 - $K=2$ **peaks then degrades** (R4 52.8 $\rightarrow$ 48.6; R0 collapses)
 - $K=8$ **improves monotonically** to the final step
- $\Rightarrow$ Over-optimisation is a $K=2$ **phenomenon**
— exactly what the lower $\text{Var}(\hat{g})$ predicts.

I.3 Diagnostic — the cost of $K=8$: group degeneracy



At $K=8$, about **a third of groups are degenerate** — all K rewards equal $\Rightarrow$ **zero advantage**, wasted samples (the known dead-group / effective-sample-size failure mode).

Yet within-group σ_r **holds** ≈ 1.5 (steady, *not* $\rightarrow 0$).

The net effect is still **+8–10 pp**.

I.3 Findings vs. the theory

Prediction (1) holds

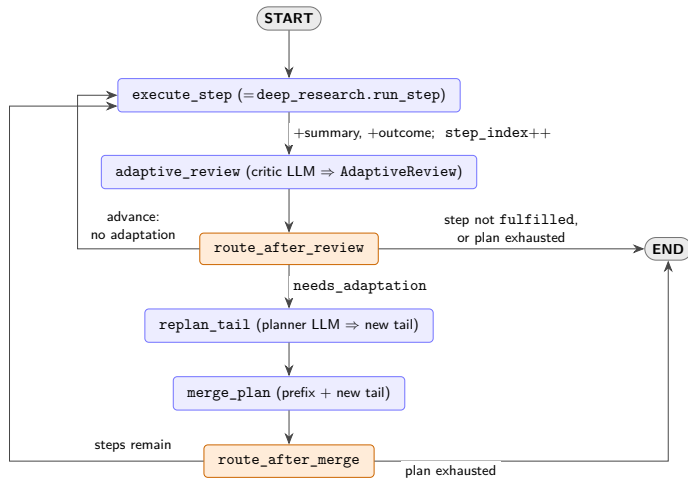
$K=8$ is the winning intervention. **Variance reduction** (group size) drives the +8–10 pp; over-optimisation is confined to $K=2$.

Prediction (2) contradicted

The tighter KL leash did *not* help: $\beta=0$ beats $\beta=0.08$ at $K=2$; $\beta=0.3$ collapses *faster*. At $K=8$ the KL weight is statistically irrelevant (R3b vs. R5: +1.7 pp, n.s.).

Punchline: the cure was **variance reduction** (larger groups), *not* regularisation — echoing **DAPO**'s removal of the KL term for long-output RL.

Part II Adaptive planning in cmbagent_lg



Walkthrough.

- **Triggers a revision:** `adaptive_review` (critic LLM) sets `needs_adaptation`; the router replans *iff* the step was fulfilled, the plan is *not* exhausted, *and* adaptation is needed.
- **What may change:** *only* the **tail** (unrun steps); `merge_plan` = prefix + new tail.
- **Held fixed:** the completed prefix, its summaries / outcomes, and `step_index`.
- **Terminates:** when a step is not fulfilled or the plan is exhausted.

Part II Problems → proposed fixes

Problem	Fix
P1 Termination not <i>guaranteed</i> — the replanned tail can grow unboundedly; no monotone-decreasing quantity; <code>recursion_limit</code> <i>raises</i> , not finishes.	→ Thread a revision budget / max-steps ; routers finish gracefully; reject over-long tails. (<i>A hard cap can truncate a genuine expansion.</i>)
P2 Adaptation only on <i>success</i> — a <code>fulfilled=False</code> step routes straight to END; no retry or route-around.	→ Add a recovery / repair path on failure with a per-step retry budget. (<i>Needs a recoverable-vs-fatal flag.</i>)
P3 Replanned tail <i>bypasses</i> the reviewer that gates the initial plan — <code>merge_plan</code> splices it unchecked.	→ Route <code>new_tail</code> through the existing plan_reviewer for one critique round. (<i>One extra reviewer call.</i>)
P4 Partial-state feedback at unconditional cost — sees only the <i>last</i> step (Markov); fires every step.	→ Full-history, gated review — cheap model / skip when no downstream impact. (<i>Gate conservatively.</i>)